

main1 Branch0 Tags

Go to fileTGo to fileAdd fileCode

 ansar-butt UGHHH	aa2622d · 1 minute ago	
 .DS_Store	Initial commit	3 years ago
 .gitattributes	Initial commit	3 years ago
 .gitignore	Modifications	last year
 MCG.txt	Commit	last year
 README.md	UGHHH	1 minute ago
 Source.cpp	Commit	last year
 TAC.txt	Commit	last year
 input.cc	Commit	last year
 output.txt	Restructuring	last year
 strings.txt	Commit	last year
 symbolTable.txt	Commit	last year

Lecture 1

1. Applications of Deep Learning

Machine learning is utilized in almost all fields of research. Your slides categorize deep learning applications into several primary domains:

Image Processing

- **Classification:** The goal is to learn to distinguish between different classes using labeled examples. Challenges include a large variability of viewpoints, multiple objects in a single image, unseen classes, and sub-categories.
- **Object Detection:** This involves predicting bounding boxes to locate multiple instances of objects within an image and classifying them. Issues arise with defining object classes and drawing accurate bounding boxes for non-square objects.
- **Face Recognition:** Often used to protect devices (authentication/verification) or assign a name to a person (identification). It involves network training on massive datasets, enrollment via transfer learning, and probing to compare faces.

Text Processing

- **Text Completion:** Predicts the next letter or word by learning from unlabeled data. Challenges include distinguishing letters versus words, special characters, and misspelled words.
- **Text Translation:** A sequence-to-sequence task that learns from paired data across a source and target language. It struggles with different sequence orders, grammar rules, and a lack of training data for rare languages.
- **Text Generation:** Also a sequence-to-sequence task, but it leverages massive amounts of unlabeled data. Issues include generating plausible but factually incorrect text, inheriting biases from training data, and guessing the context of ambiguous queries.

Sound and Other Domains

- **Music Recognition:** Aims to identify a song's title and artist by processing high-frequency sound data and extracting features. Challenges include background noise and cover songs.
- **Speech Recognition:** Transcribes spoken text by learning from paired data. It must handle different speakers, overlapping voices, varying dialects, and poor speech quality.
- **Other Applications:** Deep learning is also used for market prediction (numerical data), recommender systems (graph data), physiological signals (time series data), sports classification (video data), tumor classification from medical scans, and physics modeling.

2. Machine Learning Paradigms: What to Learn

Your lecture outlines three primary data learning tasks, split between Unsupervised and Supervised learning.

Unsupervised Learning

- **Dimensionality Reduction:** This process takes high-dimensional input and learns an embedding f_θ to keep only the important information. It relies entirely on unlabeled input images.

Supervised Learning

- **Classification:** Uses labeled categorical data in a feature space to learn a decision boundary. Once the decision boundary is learned, training samples are discarded, and the model is applied to unseen data.
- **Regression:** Aims to approximate data featuring continuous target values t by finding a function $t \approx f_\theta(\vec{x})$. Regression must account for noisy data. A simple linear regression model assumes linear data, whereas more complex models are required for non-linear data. A major risk in complex modeling is creating an "Overfit Model" that merely memorizes the training data rather than learning the underlying pattern.

3. The Single-Layer Perceptron

The Perceptron is a foundational algorithm developed by Widrow & Hoff (1960) and Rosenblatt (1962) to mimic a biological neuron using binary inputs and outputs. It essentially combines an "Adaline" (Adaptive Linear Neuron) with a threshold activation function.

Mathematical Definition

- **Activation:** The initial mathematical step is the linear combination of inputs and weights, formulated as $a = \sum_{d=1}^D w_d \cdot x_d + b$.
- **Vector Notation:** By defining $x_0 = 1$ and $w_0 = b$, the equation simplifies to $a = \sum_{d=0}^D w_d \cdot x_d$, which is written in vector notation as $a = \vec{w}^T \vec{x}$.
- **Bias (b):** The bias acts as the intercept for when $\vec{x} = 0$. Without a bias, if all input features x_d are zero, the activation naturally becomes zero, limiting the model's flexibility.
- **Output / Activation Function:** The final prediction y uses a sign activation function, defined as $y = g(a)$. Specifically, the output is +1 if $a \geq 0$, and -1 otherwise.

The Perceptron Learning Algorithm

The algorithm learns how to select optimal weights automatically through the following steps:

1. Randomly initialize the weights $\vec{w}$.
2. Choose a training sample composed of the input and the target label $(\vec{x}, t)$.
3. Predict its class using the activation function $y = g(\vec{w}^T \vec{x})$.
4. Check if the prediction is wrong, which occurs when $y \cdot t < 0$. If it is incorrect, update the weights using the rule: $\vec{w} = \vec{w} + t \cdot \vec{x}$.

Understanding the Update Rule:

□ README

always be negative, triggering the update.

- If $y = -1$ but the actual target is $t = 1$, the rule adds the input vector $\vec{x}$ to the weights $\vec{w}$.
- If $y = 1$ but the actual target is $t = -1$, the rule subtracts the input vector $\vec{x}$ from the weights $\vec{w}$.



Advantages vs. Disadvantages

- **Advantages:** The algorithm is simple, quick, and accommodates continuous inputs. Crucially, it is mathematically guaranteed to converge if the inputs are linearly separable.
- **Disadvantages:** It is not easily extendable and yields a non-optimal solution rather than the "best" possible boundary. It only outputs binary predictions, and if the data is *not* linearly separable, the algorithm gets trapped in an endless loop.

4. Loss Functions

Loss functions (also known as Error functions $E(X)$ or Cost functions C) measure the quality of algorithms over a dataset X . They compare the model's output to the target labels to provide a means of learning. By definition, lower values indicate better model performance.

General Notation

A dataset X contains N total samples, written as $X = (\vec{x}^{[1]}, t^{[1]}), (\vec{x}^{[2]}, t^{[2]}), \dots, (\vec{x}^{[N]}, t^{[N]})$. The generic loss function parameterized by θ is formulated as $J_{\theta}(X) = \sum_{n=1}^N \text{cmp}(f_{\theta}(\vec{x}^{[n]}), t^{[n]})$.

The Perceptron Loss Function

For two binary classes where $t^{[n]} \in \{-1, +1\}$, the specific Perceptron Loss is defined as:

$$J_{\text{Perc}}(\vec{w}) = - \sum_{n=1}^N \begin{cases} t^{[n]} \vec{w}^T \vec{x}^{[n]} & \text{if } y^{[n]} t^{[n]} < 0 \\ 0 & \text{else} \end{cases}$$

- The loss evaluates to 0 for correctly classified samples.
- The loss returns a positive value for incorrectly classified samples.

Proof Sketch: Why the Weight Update Works

Your slides provide a mathematical proof sketch showing that applying the update rule $\vec{w} = \vec{w} + t\vec{x}$ strictly lowers the overall loss.

- Evaluating the new weight vector for a misclassified sample yields: $-t(\vec{w} + t\vec{x})^T \vec{x}$.
- Expanding this equation results in: $-t\vec{w}^T \vec{x} - t\vec{x}^T t\vec{x}$.
- This expanded equation is strictly less than the original loss $-t\vec{w}^T \vec{x}$.
- This inequality holds true because the term $t^2 \vec{x}^T \vec{x} > 0$ for all t and $\vec{x}$, given that the bias term $x_0 = 1$ ensures the vector is never completely zero.

Lecture 2

1. Linear and Logistic Regression

Before diving into multi-layered neural networks, the lecture grounds the fundamentals in single-neuron regression models.

- **Linear Regression:** The goal here is to fit a continuous function to data points. It relies on calculating the Mean Squared Loss, represented by the formula $J(\vec{w}) = \frac{1}{2N} \sum_{n=1}^N (y^{[n]} - t^{[n]})^2$.
- **Logistic Regression:** While linear regression handles continuous outputs, logistic regression is used for more complex, non-linear approximations, mapping outputs to a range between 0 and 1. It does this using a logistic (or sigmoid) activation function $g(a) = \frac{1}{1 + e^{-a}}$.
- **The Limitation:** Both linear and logistic regression models act as a single line (or single neuron), meaning they can only map linearly separable problems. They completely fail to solve inherently non-linear datasets, such as the classic XOR problem.

2. Gradient Descent

Because a closed-form solution doesn't always exist for models like logistic regression, we use an iterative optimization algorithm called Gradient Descent.

- **The Goal:** To find the optimal parameters (weights) by minimizing the loss function, which essentially means finding the lowest point on the loss surface.

- **The Mechanism:** The gradient mathematically points toward the steepest *increase* in the loss function. Therefore, to minimize the loss, the network updates its weights by taking a step in the direction of the *negative* gradient.
- **Weight Update Rule:** This update process is represented by the formula $\vec{w} = \vec{w} - \eta \nabla_{\vec{w}}(J)$. Here, η represents the learning rate, or "step size".
- **Potential Pitfalls:** It is not a flawless system. If your learning rate η is too small, the network might get stuck in a local minimum, a plateau or a saddle point. If η is too large, the algorithm might jump completely over the global minimum and diverge.

3. Two-Layer Networks & Activation Functions

To overcome the limitations of a single neuron (like the inability to solve the XOR problem), the architecture is expanded into multiple layers.

- **Model Capacity:** Adding a hidden layer with multiple neurons equates to adding multiple lines or hyperplanes, drastically increasing the model's mathematical capacity.
- **The Need for Non-Linearity:** If you simply stack linear layers, the math mathematically collapses back into a single linear neuron. To prevent this, non-linear activation functions must be applied to the hidden neurons.
- **Universal Approximation Theorem:** This theorem dictates that a neural network with just one hidden layer, utilizing non-linear activation functions and a sufficient number of neurons, can approximate any continuous function to any reasonable accuracy.
- **Activation Function Rules:** To function correctly in backpropagation, these activation functions must be continuous, differentiable, and monotonic. Common examples include the Sigmoid function $\sigma(a)$ and the Hyperbolic tangent function $\tanh(a)$.

4. Error Backpropagation

This is how the network actually "learns" across multiple layers. It is the process of calculating the gradient of the loss with respect to all of the parameters θ inside the network.

- **The Chain Rule:** Backpropagation relies heavily on the chain rule of calculus: $\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(x)}{\partial g(x)} \frac{\partial g(x)}{\partial x}$. This allows the network to link together the gradients of concatenated functions layer by layer.
- **Second Layer Gradients:** To update the weights closest to the output $\vec{w}^{(2)}$, the chain rule calculates the derivative of the loss with respect to the output, multiplied by the derivative of the output with respect to the weights.
- **First Layer Gradients:** To update the initial hidden layer weights $W^{(1)}$, the math continues backward. It chains the output error through the hidden layer's activation function to determine exactly how those early weights contributed to the final loss.

Lecture 3

1. Foundations & The Two-Layer Network Architecture

This section reviews basic feedforward mechanics and error minimization using backpropagation.

Gradient Descent & Optimization

- **Core Principle:** To minimize a loss function J , the model evaluates the gradient $\nabla_{\vec{w}} J$ at the current weight configuration $\vec{w}$ and takes a step in the direction of the steepest descent (the negative gradient).
- **Learning Rate (η):** This scalar controls the step size of each update.
- **Iterative Weight Update:** The mathematical framework repeats the following adjustment epoch after epoch:

$$\vec{w} = \vec{w} - \eta \nabla_{\vec{w}}(J)$$

Network Components & Function Chains

The architecture maps inputs to outputs through sequential layers:

- **Pre-activations (a_k):** The linear combination of inputs and weights ($W^{(1)} \vec{x}$).
- **Activation Functions:** Non-linear transformations map pre-activations to hidden states (h_k). The slides contrast two logistic activation options:
- **Sigmoid:** $\sigma(a) = \frac{1}{1 + e^{-a}}$, squashing values between (0, 1).

- **Hyperbolic Tangent:** $\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$, mapping values between $(-1, 1)$.
- **The Function Chain:** Represented as a composite mathematical sequence: $y = \vec{w}^{(2)T} g(W^{(1)} \vec{x})$.
- **Mean Squared Loss (L_2 Loss):** Measures empirical performance across N training samples:

$$J_{\theta}^{L_2} = \frac{1}{N} \sum_{n=1}^N (y^{[n]} - t^{[n]})^2$$

Backpropagation via the Chain Rule

To calculate how much an internal weight contributes to the overall loss, the network passes errors backward layer-by-layer:

- **Second Layer Gradient:** Directly measures the output error variance relative to the second-layer weights:

$$\frac{\partial J}{\partial w_k^{(2)}} = \frac{1}{N} \sum_{n=1}^N \frac{\partial J^{[n]}}{\partial y^{[n]}} \frac{\partial y^{[n]}}{\partial w_k^{(2)}}$$

- **First Layer Gradient:** Leverages a deeper chain rule composition to find dependencies through the activation function's derivative (f') and the initial input layer:

$$\frac{\partial J}{\partial w_{k,d}^{(1)}} = \frac{1}{N} \sum_{n=1}^N \frac{\partial J^{[n]}}{\partial y^{[n]}} \frac{\partial y^{[n]}}{\partial h_k^{[n]}} \frac{\partial h_k^{[n]}}{\partial a_k^{[n]}} \frac{\partial a_k^{[n]}}{\partial w_{k,d}^{(1)}}$$

2. Optimization Paradigms: GD vs. SGD vs. Batched SGD

The slides use a non-linear sinusoidal regression task to demonstrate how modifying optimization data structures radically speeds up convergence.

Feature	Vanilla Gradient Descent (GD)	Stochastic Gradient Descent (SGD)	Mini-Batch SGD
Data per Update	Entire training set X	Single random sample $(\vec{x}^{[n]}, t^{[n]})$	A small block/subset of data of size B
Update Stability	Extremely stable; small, averaged updates	High volatility; weights and loss fluctuate wildly	Balanced; stabilizes gradient directions
Parallelization	Low optimization performance per step	Non-parallelizable	Highly parallelizable via matrix acceleration
Streaming Data	Requires a fixed, static dataset	Supports continuous data streams	Supports split batch processing over streams

Why Traditional Stopping Criteria Fail in SGD/Batched SGD

- **The Problem:** In classic GD, optimization stops when the change in total loss ($\Delta J(X) < \epsilon$) or the gradient norm ($\| \nabla_{\theta} J(X) \| < \epsilon$) falls below a threshold. Because SGD and Batched SGD evaluate sub-allocations rather than the full dataset at each step, these global metrics are missing.
- **The Solution:** Training is bound to a fixed number of predetermined epochs or iterations as the default structural fallback.

3. Optimized Matrix Implementation

To bypass the severe computational bottlenecks of native Python indexing loops, the lecture emphasizes converting algebraic operations into vectorized linear algebra handled by optimized libraries (like MKL).

- **Memory Allocation Optimization:** Reassigning data blocks continually allocates memory and triggers copy sequences in Python. Using in place transformations like `y += x` or exploiting NumPy's `out` variable (e.g., `numpy.add(x1, x2, out=y)`) optimizes performance.
- **Block Forward Pass Processing:** Instead of processing single vectors, entire batches are combined into an input matrix $X \in \mathbb{R}^{(D+1) \times B}$ and parsed concurrently:

$$H = g(W^{(1)}X) \Rightarrow \vec{y} = H^T \vec{w}^{(2)}$$

- **Matrix Gradients:** Computations use the Hadamard element-wise product ($\odot$) and the matrix outer product ($\otimes$) to derive updates for thousands of components at once:

$$\nabla_{W^{(1)}} = \frac{2}{B} [(\vec{w}^{(2)} \otimes (\vec{y} - \vec{t})) \odot H \odot (1 - H)] X^T$$

4. Momentum Learning

To counteract the oscillating path modifications typical of sample-wise updates, momentum learning acts as a physical stabilizer.

- **Basic Concept:** It tracks the direction of the immediately preceding weight update ($\theta^r - \theta^{r-1}$) and incorporates a fraction μ of it into the current step. This keeps the parameter trajectory moving smoothly along reliable down-slope paths.
- **The Mathematical Adjuster:**

$$\theta^{r+1} = \theta^r - \eta \nabla_{\theta} J + \mu(\theta^r - \theta^{r-1})$$

- **Hyperparameters:** The momentum coefficient μ is typically set to high-density thresholds like 0.5, 0.9, or 0.99.

5. Multi-Output Networks

When dealing with multidimensional target spaces, the network scales from a single output scalar y to an output vector $\vec{y} \in R^O$.

Structural and Inter-Layer Dependencies

- **Forward Pass Mapping:** Every singular output node y_o explicitly tracks its own row-specific second-layer weights $w_{o,k}$ but simultaneously relies on every first-layer weight allocation $w_{k,d}$.
- **Backward Error Allocation:** While second-layer weights $w_{o,k}$ rely strictly on the error generated at their designated output node y_o , updating any first-layer weight $w_{k,d}$ requires summing the backpropagated errors across **all** output channels.

Frobenius Norm for Vectorized Loss

To quantify multi-dimensional errors across a complete mini-batch, the objective function shifts from standard scalar operations to the Matrix Frobenius Norm ($\| \cdot \|_F^2$):

$$J_{\theta}^{L_2} = \frac{1}{B} \| Y - T \|_F^2$$

6. Stabilization, Normalization & Regularization

Deep architectures are highly vulnerable to vanishing gradients and out-of-scale variations. The lecture highlights three critical safeguards to maintain stable training:

Data Normalization

- **Input Normalization:** If input dimensions have wildly different scales, features with large absolute values dominate the gradients. Standardizing inputs (subtracting mean μ_d and dividing by standard deviation σ_d) keeps feature influence balanced.
- **Target Normalization:** Massive target targets (t_o) inflate the loss, requiring an impractically small learning rate. Standardizing targets during training—and reversing the transformation before making final predictions ($y_o \leftarrow y_o \cdot \sigma_o + \mu_o$)—stabilizes the gradient steps.

Guided Weight Initialization

- **The Trap:** If initialized weights are too large, the pre-activations land in saturation zones ($|a| > 5$), where the derivatives of sigmoid or tanh approach zero, halting learning. If they are too small ($|a| \approx 0$), the activations behave linearly, collapsing the multi-layer network into a simple linear model.
- **The Solution (Xavier/Glorot Initialization):** Weights are sampled uniformly from a dynamic range based on layer size:

$$s = \frac{\sqrt{6}}{\sqrt{D+K}}$$

This preserves a variance of 1 across layers, avoiding saturation and enabling larger initial learning rates.

Weight Decay (L_2 Regularization)

- **Mechanism:** To prevent weights from exploding during training, a penalty term proportional to the squared magnitude of the weights ($\lambda ||\theta||^2$) is added to the objective function.
- **Regularized Gradient:**

$$\nabla_{\theta} = \frac{\partial J^{total}}{\partial \theta} = \frac{\partial J}{\partial \theta} + 2\lambda\theta$$

This addition continuously penalizes large weights, pulling them back toward zero and keeping the model out of saturation zones.

Lecture 4

1. Binary Classification

Binary classification deals with tasks where there are only two possible outcomes, such as distinguishing between cats and dogs, or determining if code is malicious or safe. The target variable is restricted to $t \in \{0, 1\}$.

The Network Structure and Probabilistic Interpretation

To perform binary classification, you typically start with a standard neural network ending in a logit output $z = w^{(2)T}h$.

- An output activation function $g^{(2)}$ is added to map this logit into a probability score $y \in [0, 1]$.
- This relies on a Bernoulli distribution interpretation.
- The network's output y represents the probability that the positive class is true, i.e., $P(t = 1) = y$.
- Consequently, the probability of the negative class is $P(t = 0) = 1 - y$.

Binary Cross-Entropy (BCE) Loss

To train the network, we must maximize the probability of correctly classifying the entire dataset. Because taking the product of probabilities is mathematically intractable and numerically unstable, we take the negative logarithm, resulting in the **Binary Cross-Entropy (BCE) Loss** (also known as Negative Log-Likelihood):

$$J^{BCE} = - \sum_{n=1}^N [t^{[n]} \log y^{[n]} + (1 - t^{[n]}) \log(1 - y^{[n]})]$$

The Logistic (Sigmoid) Activation Function

To make the BCE loss easy to compute (and to cancel out the natural logarithms), the logistic (or sigmoid) function is chosen as the activation function:

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The "Magic" of the Gradient

A crucial revelation in the slides is the calculation of the gradient of the BCE loss with respect to the pre-activation logit z . Due to the mathematical relationship between the BCE loss and the logistic function (via the SoftPlus function properties), the derivative simplifies elegantly:

$$\frac{\partial J^{[n]}}{\partial z^{[n]}} = y^{[n]} - t^{[n]}$$

This derivative is identical to the derivative used in standard Mean Squared Error (J^{L_2}) for regression tasks, meaning the underlying backpropagation mechanics remain largely identical despite the shift to classification.

2. Categorical Classification

Categorical (or multi-class) classification extends the binary concept to O mutually exclusive classes. Instead of a single output, the network now uses a multi-output architecture.

SoftMax Activation

The network computes a vector of logits z which are passed through a SoftMax activation function to yield a vector of probabilities y :

$$y * o = \frac{e^{z_o}}{\sum_{o'=1}^O e^{z_{o'}}}$$

- The SoftMax function ensures that all outputs sum exactly to 1, creating a valid probability distribution across all classes.
- It is sensitive to the *differences* between logits rather than their absolute values.

Categorical Cross-Entropy (CCE) Loss and One-Hot Encoding

To compute the loss, target labels are transformed into **One-Hot Vectors**, where the correct class index $\tau^{[n]}$ is 1 and all other indices are 0. For example, if class 3 out of 4 is correct, the target is (0, 0, 1, 0).

The Categorical Cross-Entropy loss heavily penalizes the network if it assigns a low probability to the true class:

$$J^{CCE} = -\frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O t_o^{[n]} \log y_o^{[n]}$$

Strikingly, just like in binary classification, the derivative of the CCE loss with respect to the logit z_o simplifies to $(y_o - t_o)$. The SoftMax activation essentially turns a regression architecture into a categorical classification architecture.

3. Evaluation and Early Stopping

While loss functions are essential for gradient descent, they are difficult for humans to interpret. Therefore, models are evaluated using **Accuracy**—the percentage of samples classified correctly.

- **Binary Accuracy:** Threshold the output probability y at 0.5 (or the logit z at 0).
- **Categorical Accuracy:** The predicted class is the one with the highest probability, utilizing the `argmax` function.

Mitigating Overfitting

Neural networks can easily memorize the training data, creating bizarre decision boundaries that fail on new data—a phenomenon known as overfitting.

- **Validation Set:** Typically, 20% of the data is split off and left completely unseen during training.
- **Early Stopping:** After each training epoch, the model's loss and accuracy are evaluated on this validation set.
- If the training loss continues to drop but the validation loss begins to increase, the model is overfitting.
- Training is halted when the validation model shows no improvement over several epochs, and the weights that yielded the lowest validation loss are restored.

4. Unification of Nomenclature

The lecture concludes by harmonizing these different architectures. Whether you are doing regression, binary classification, or categorical classification, the core framework is the same. The only difference is the final activation function applied to the logits:

- **Regression:** Uses an Identity activation $y = I(z) = z$.
- **Binary Classification:** Uses the Logistic/Sigmoid activation $y = \sigma(z)$.
- **Categorical Classification:** Uses the SoftMax activation $y = S(z)$.

Lecture 5

1. Fundamentals: Review and Automatic Differentiation

Before introducing convolutional architectures, the lecture establishes how neural networks compute gradients to learn.

- **Binary and Categorical Classification:** Standard networks use a logit function, applying a Sigmoid activation for binary classification (using Binary Cross-Entropy loss) or a SoftMax activation for categorical classification (using Categorical Cross-Entropy loss).
- **Computation Graphs:** To automate backpropagation, networks use computation graphs where mathematical operations act as nodes.

- **Graph Traversal:** The backward pass starts at the end of the graph and traverses backward, multiplying partial derivatives via the chain rule to compute gradients for updating weights.
- **Vectorized Differentiation:** Instead of computing derivatives for single variables, modern frameworks compute Jacobian matrices (matrix of partial derivatives).
- **Jacobian Matrix:** For a function mapping multiple inputs to multiple outputs, the Jacobian matrix systematically organizes all partial derivatives, allowing the network to use efficient matrix multiplications for large derivative chains.

2. The Shift to Convolutional Networks

Traditional "Fully-Connected" dense layers are poorly suited for image processing. Convolutional layers solve many of the inefficiencies inherent to processing visual data.

Feature	Fully-Connected Layers	Convolutional Layers
Parameter Count	High, consuming substantial memory	Lower, making them less prone to overfitting
Input Dependencies	Assumes all input dimensions are independent	Models spatial dependencies between input dimensions
Input Size	Requires fixed input and output sizes	Can handle variable input dimensions

3. Mechanics of Convolution

A convolutional layer applies a "kernel" (a small grid of weights) that slides across the input data to detect features.

- **Discrete Convolution (1D & 2D):** The kernel multiplies its weights against a localized region of the input and sums the result to produce a single output value in a new "feature map".
- **Zero Padding:** This technique adds borders of zeros around the input data, ensuring the spatial dimensions of the output match the input though it can introduce artificial borders.
- **Stride:** This involves moving the kernel multiple steps at a time to reduce the output dimensions, though it risks skipping important values.

4. Pooling Layers

Instead of using strides to reduce spatial dimensions, networks typically use Pooling layers. Pooling operations incorporate several outputs from a defined region into a single value.

- **Average Pooling:** Calculates the average value within the defined region.
- **Maximum Pooling:** Extracts the highest value within the defined region, which is the most commonly used variation.

5. Multi-Dimensional and Stacked Convolutions

To process real-world images, the convolutions must expand beyond simple 2D grids.

- **Color Channels:** Color images are 3D tensors, denoted formally as $X \in \mathbb{R}^{C \times D \times E}$, where C represents the channels (e.g., 3 for Red, Green, Blue).
- **Multiple Convolutions:** A single filter is insufficient to detect complex patterns, so networks learn multiple filters to detect various scales and orientations, resulting in a 3D output known as a feature map.
- **Stacked Convolutions:** Individual kernels only "see" a few pixels at a time (their receptive field).
- **Receptive Field Expansion:** By stacking multiple convolutional and pooling layers, deeper layers can recognize larger and more complex structures without a quadratic explosion in parameter count.

6. Architecture and Best Practices

The end of a convolutional network usually transitions back to fully-connected layers to output a final classification.

- **Flattening:** This transforms a multi-dimensional feature map into a 1D vector by mathematically reinterpreting the 3D index into a 1D index, requiring no actual data copying in memory.
- **Global Pooling:** Alternatively, reducing the entire spatial dimension of a feature map into a single value, ensuring the output is independent of the original image resolution.
- **Kernel Sizes:** Common kernel sizes are 3x3 or 5x5.
- **Activations:** The ReLU function is generally the preferred activation function.
- **Network Depth:** Deep networks (more layers) are generally favored over wide networks (larger or more kernels).
- **The Rule of Ten:** As a standard rule for machine learning, you generally need at least 10 times more training data than you have parameters.
- **LeNet-5 (1989):** A classic example provided is the LeNet architecture, which successfully stacked convolutions, average pooling, and fully connected layers to classify handwritten digits (the MNIST dataset).

Lecture 6

1. Review of Fundamentals

Before diving into deep networks, the slides review the mathematical building blocks of Convolutional Neural Networks (CNNs).

- **Convolutions and Pooling:** The network uses 1D, 2D, and channeled convolutions to process inputs (like images) using kernels (filters) to produce output feature maps. Pooling layers are then used to downsample these outputs.
- **LeNet Topology:** An early CNN architecture example, LeNet, is shown using a sequence of 5x5 convolutions, average pooling, and fully connected (FC) layers to reach a final classification.
- **Calculus in Deep Learning:** Network training relies heavily on the Jacobian Matrix ($J_f = \frac{\partial \vec{y}}{\partial \vec{x}}$), which represents the matrix of partial derivatives. This allows for vectorizing gradient descent across layers via the chain rule to update weights.

2. Deep Networks

This section addresses why stacking more layers is beneficial and introduces the foundational deep architectures that defined the field.

- **Why Go Deeper?:** Convolutional kernels have small receptive fields (typically 3x3 or 5x5 pixels). Stacking layers increases this receptive field, allowing the network to learn more complex features. Increasing the depth is heavily preferred over simply increasing kernel sizes or channel counts, which would cause a quadratic increase in computational requirements.
- **The ImageNet Challenge (ILSVRC 2012):** This benchmark dataset tasked models with classifying high-resolution images into 1,000 distinct classes. Performance is evaluated using Top-1 Accuracy (the model's most confident prediction is correct) and Top-5 Accuracy (the true label is within the model's five most confident predictions).
- **AlexNet:** This was the first deep convolutional network to win the ImageNet challenge by a large margin. It introduced the ReLU activation function, DropOut regularization, and extensive data augmentation. However, it possessed 61 million parameters, used very large 11x11 input filters, and was prone to overfitting.
- **VGG Network:** To improve upon AlexNet, VGG-19 utilized deeper stacks of smaller 3x3 convolutions. Despite a massive parameter count of 144 million, it achieved a highly superior Top-5 accuracy of 92.0% on the ImageNet validation set.

3. Going Deeper: Challenges and Solutions

As networks grow deeper, they become harder to train. The slides outline several critical innovations designed to stabilize and accelerate the training of deep models.

Regularization via DropOut

- To avoid overfitting, DropOut acts as a form of "model bagging" within a single network.
- During training, it randomly disables (drops) single neurons in fully connected layers with a set probability κ . This forces the network to distribute knowledge across all nodes rather than relying on a few "master neurons". During testing, the entire network is used.

Vanishing and Exploding Gradients

- **The Problem:** In very deep networks using Sigmoid or Tanh activation functions, the gradient often vanishes (shrinks to zero) in early layers during backpropagation because their maximum derivatives are very small (0.25 and 1, respectively). Exploding gradients occur when weight values become excessively large.
- **The Solution (ReLU):** The Rectified Linear Unit ($\text{ReLU}(a) = \max(0, a)$) solves the vanishing gradient problem because its derivative is simply 1 for all positive inputs. Exploding gradients are usually countered using Weight Decay.

Residual Networks (ResNet)

- Historically, simply adding depth to a network did not guarantee higher quality or accuracy.
- ResNet solved this by introducing **Residual Blocks** with shortcut connections. Instead of a layer only learning a direct mapping, it adds its original input back to its output: $A^{(l+1)} = W^{(l)} * A^{(l)} + A^{(l)}$.
- Because the derivative of the addition operator is 1, gradients flow easily through the network, allowing highly successful architectures up to 152 layers deep.

Batch Normalization & Adam Optimizer

- **Batch Normalization:** When weights update in deep networks, the distribution of inputs to subsequent layers shifts drastically. Batch Normalization stabilizes this by normalizing inputs using the batch's mean and standard deviation. It introduces learnable shift ($\vec{\beta}$) and scale ($\vec{\gamma}$) parameters so the network retains the ability to model complex distributions.
- **Adam Optimizer:** While standard Gradient Descent applies the same learning rate to all parameters, Adam computes an adaptive learning rate per parameter. It uses running averages of the gradient's mean (first moment) and variance (second moment) to intelligently optimize weights.

4. Transfer Learning & Representation Learning

Because deep networks require massive amounts of labeled data, training from scratch isn't always possible.

- **Domain Adaptation:** You can pre-train a network on a massive dataset (like ImageNet) and then transfer that knowledge to a smaller dataset. This is done by freezing the early layers (which detect general features like edges) and fine-tuning the last layers to predict new, specific classes.
- **Representation Learning:** Alternatively, you can drop the final classification layer entirely and use the deep network purely as a feature extractor. The network processes an image and outputs a highly descriptive numerical vector ($\vec{\phi}$).
- **Deep Face Recognition:** The slides use Face Recognition as a prime example of representation learning. Because it is impossible to train a classifier with a dedicated node for every single person on Earth, networks are pre-trained on huge celebrity datasets to extract deep facial features. Loss functions like **CosFace** are used to mathematically force the features of the same person tightly together in the feature space while pushing the features of different people apart by incorporating a cosine margin m . Afterwards, to verify a person's identity (e.g., unlocking a phone), the system simply calculates the cosine similarity between the deep features of the new face and a stored reference face.

Lecture 7

1. The Open-Set Problem Statement

Traditional neural network training relies on a closed-set view of the world.

- Closed-set classification assumes the model will only ever encounter the O specific classes it was trained to separate in the feature space.
- However, the real world operates in an open space, meaning unforeseeable, unknown samples will inevitably be presented to the model.
- Standard classifiers are forced to assign these unknown samples to one of the known classes, and they typically do so with extremely high confidence.
- For instance, an object detector trained strictly on horses, cows, and sheep might confidently detect an unseen elephant and misclassify it as a "cow" with a 0.96 confidence score.
- The ultimate goal of Open-Set Recognition is to equip the classifier with an "I don't know" option rather than forcing an incorrect assignment.

2. Deep Feature Visualization & The SoftMax Flaw

To understand why standard networks fail at rejecting unknowns, the lecture introduces the concept of a "Bottleneck Network".

- A Bottleneck Network (such as LeNet++) intentionally restricts deep features, denoted as $\vec{\phi}$, to a 2D coordinate system (ϕ_1, ϕ_2) before the final fully-connected layers.
- This constraint allows researchers to plot and visualize the deep feature representation of datasets, revealing how classes cluster (e.g., MNIST digits forming a 10-lobe flower structure).
- When visualizing the geometric interpretation of the final layer, the decision boundaries are calculated as 2D planes mathematically defined as $z_o = w_{o,0}^{(L)} + w_{o,1}^{(L)}\phi_1 + w_{o,2}^{(L)}\phi_2$.
- Because of how the SoftMax activation computes probabilities, the maximum confidence score y_o approaches 1 almost everywhere in the coordinate space.
- Consequently, even completely unseen or negative data (like Devanagari characters fed to a digit recognizer) will fall somewhere in this space and naturally receive artificially high confidence scores.

3. Training Networks for Open-Set

To properly frame the solution, the lecture splits data into three distinct class types:

Class Type	Description	Training Status
Known	Target classes that must be classified correctly.	Present during training.
Negative	"Known unknowns" that are uninteresting but available.	Present during training.
Unknown	Completely unseen and unforeseeable data.	Never seen during training.

The lecture details two primary strategies for integrating negative data into training to prepare the network for unknowns:

Background/Garbage Class

- This approach adds an extra class (totaling $O + 1$ classes) specifically designated for negative samples, where $\tau = 0$.
- Because known classes are generally balanced while negative samples are abundant, this method suffers from severe class imbalance.
- To fix this, sample weights $\lambda_\tau = \frac{N}{(O+1)N_\tau}$ must be applied to the weighted gradient during training.

SoftMax Adaptation (Entropic Open-Set Loss)

- Instead of adding a class, this method relies strictly on the default O classes and alters the target probabilities during training.
- Known classes are trained using standard one-hot encoded vectors.
- Negative samples are assigned an equal probability distribution across all classes, meaning $y_o = \frac{1}{O}$.
- This adaptation mathematically forces the network to map negative and unknown samples toward the origin ($\vec{\phi} = \vec{0}$) in the feature space.
- By clustering near the origin, unknown samples naturally yield low confidence scores across the board, allowing for confident rejection.

4. Advanced Open-Set Modeling Techniques

The slides outline two specific techniques that modify the network to better model class representations mathematically.

Open-Set Deep Networks (OSDN)

- OSDN uses deep feature similarity rather than just final logits to estimate class boundaries.
- It computes Mean Activation Vectors (MAVs), denoted as $\vec{\mu}_o$, for each class based on deep training features $\vec{\phi}$.
- By calculating class-wise distances to the MAV (D_o), the algorithm uses extreme value theory to model the probability of a sample's exclusion, Ψ_o .
- Original logits are revised, and an artificial logit z_0 for an unknown class is generated before estimating the final SoftMax probabilities.

Gaussian Hypothesis Open-Set Technique (GHOST)

- GHOST acts as an upgrade for complex feature modeling.
- Instead of a single MAV, it models deep features using a multivariate Gaussian distribution per feature dimension, utilizing both means $\vec{\mu}_o$ and standard deviations $\vec{\sigma}_o$.
- Predictions rely heavily on thresholding normalized logits, calculated as:
$$y_o = \frac{z_o}{\sum_{k=1}^K \frac{|\phi_k - \mu_{o,k}|}{\sigma_{o,k}}}$$

5. Evaluating Open-Set Recognition

Standard classification accuracy metrics are flawed in this context because they struggle to handle imbalanced data and samples that technically belong to no class.

- **Open-Set Classification Rate (OSCR) Curve:** Models are better evaluated by plotting the Correct Classification Rate (CCR) against the False Positive Rate (FPR) across a sliding threshold $\zeta \in [0, 1]$.
- **Coefficient of Variation:** Because global thresholds can result in unequal performance across different classes, the Coefficient of Variation $\mathcal{V}\{CCR(\zeta)\} = \frac{\sigma\{CCR(\zeta)\}}{\mu\{CCR(\zeta)\}}$ is utilized to measure the inequality of correct classification rates across all classes.
- **Validation Metrics:** To decide when a network should stop training, validation sets maximize target confidence. For known samples, it maximizes the confidence of the correct class y_c . For unknowns, it targets $1 - \hat{y} + \frac{1}{O}$, which peaks when the predicted confidence $\hat{y}$ correctly hits $\frac{1}{O}$.

6. Ongoing Research & Infrastructure

The end of the lecture highlights current research initiatives, most notably the creation of new open-set evaluation protocols for the ImageNet dataset. These protocols group concepts (like animals and artifacts) to define varying difficulties: Easy, Intermediate, and Hard. To standardize this rapidly developing field, the research group is also building "OpenOSR," an open-source toolbox for Open-Set Recognition.

Lecture 8

1. Motivation for Unsupervised Representation Learning

Historically, extracting meaningful feature representations required heavily supervised learning (which relies on large amounts of labeled data or hand-crafted features (which must be manually tailored to fit a specific machine learning algorithm)).

However, unlabeled data is much more abundant and easier to obtain. Unsupervised learning aims to automatically extract significant information and learn the underlying significance of the data without relying on human annotations. While traditional unsupervised methods like Principal Component Analysis (PCA) can compress data, they are limited to linear transformations and struggle to handle large datasets effectively. Deep learning solves this by using neural networks to learn complex, non-linear representations automatically.

2. Auto-Encoders (AE)

Auto-Encoders are a type of auto-associative network where the network's goal is to output an exact reconstruction of its input: $\vec{x} = f_{\theta}(\vec{x})$.

To prevent the network from simply memorizing the data and acting as an identity function, Auto-Encoders utilize a "bottleneck" hidden layer where the number of hidden units (K) is significantly smaller than the input dimensions (D). This forces the network to compress the data, learning a dense "deep feature" representation in the process.

Architecture

An Auto-Encoder is typically divided into two subnetworks:

- **The Encoder:** Maps the high-dimensional input $\vec{x}$ to a compressed deep feature representation (or latent space) $\vec{\phi}$. Mathematically expressed as $\vec{\phi} = E(\vec{x})$.
- **The Decoder:** Reconstructs the original data $\vec{y}$ from the compressed feature representation $\vec{\phi}$. Mathematically expressed as $\vec{y} = D(\vec{\phi})$.
- **Structure:** The decoder often uses the inverse structural layers of the encoder. Non-linear activation functions (like ReLU or sigmoid) are applied after the hidden layers to allow the network to learn non-linear embeddings.

The Loss Function

Because the goal is to make the output $\vec{y}$ as close to the input $\vec{x}$ as possible, simple auto-encoders optimize using the Mean Squared Error (MSE) loss function:

$$J^{L_2} = \frac{1}{N} \sum_{n=1}^N ||\vec{y}^{[n]} - \vec{x}^{[n]}||^2$$

Convolutional Auto-Encoders

For image processing, fully connected layers are inefficient. Instead, Convolutional Auto-Encoders use convolution and pooling layers in the encoder to extract features.

However, reducing the spatial dimensions of an image via max-pooling creates a problem for the decoder, which must eventually reconstruct the original image size.

- **Inverse Maximum Pooling:** One solution is for the pooling layer to store the spatial "indexes" of the maximum values. The decoder then places the reconstructed values back into those specific stored locations, leaving the rest empty.
- **Fractionally-Strided Convolutions:** A more robust replacement for pooling and unpooling is the use of strided convolutions to compress the image in the encoder, and fractionally-strided convolutions (often referred to as transposed convolutions) to increase the size in the decoder. Transposed convolution inverts the standard convolution matrix operation ($\vec{a} = \tilde{W} \vec{x}$) using the transposed matrix $\vec{y} = \tilde{W}^T \vec{a}$.

3. Variations of Auto-Encoders

To enforce specific behaviors or make the feature space more robust, standard Auto-Encoders can be modified.

Sparse and Denoising Auto-Encoders

- **Sparse Auto-Encoders:** These networks apply a sparsity constraint on the bottleneck representation $\vec{\phi}$. By adding a regularization term $\Omega(\vec{\phi}^{[n]})$ to the MSE loss function, or by using techniques like Dropout or ReLU before the deep feature layer, the network is forced to activate only a small number of neurons at any given time.
- **Denoising Auto-Encoders:** These networks intentionally corrupt the input data with noise ($\vec{x}' = \vec{x} + noise$) and force the network to output the original, uncorrupted data $\vec{x}$. Because a different noise pattern is applied in every epoch, the network is forced to learn the underlying distribution of the data rather than just memorizing pixel values.

Variational Auto-Encoders (VAEs)

While standard Auto-Encoders learn discrete, sometimes disjointed feature representations, VAEs ensure the feature space is mathematically "smooth". This means that if two latent vectors $\vec{\phi}$ are close to each other, their decoded outputs will look similar.

- Instead of encoding an input into a single fixed vector, VAEs encode the input into a normal probability distribution characterized by a mean ($\vec{\mu}$) and a standard deviation ($\vec{\sigma}$).
- The deep feature $\vec{\phi}$ is then sampled from this distribution: $N_{\vec{\mu}, \vec{\sigma}}$.
- The loss function incorporates a Kullback-Leibler (KL) divergence term to ensure these distributions behave well:

$$\mathcal{J}^{VAE} = \sum_{n=1}^N [\|\vec{\phi}^{[n]} - \vec{x}^{[n]}\|^2 + \alpha \text{KL}(\mathcal{N}(\vec{\mu}^{[n]}, \vec{\sigma}^{[n]}) \parallel \mathcal{N}(\vec{0}, \vec{1}))]$$

Diffusion Models

Diffusion models are generative models that utilize an integrated encoder-decoder process across T intermediate steps.

- The forward (encoding) process iteratively adds Gaussian noise with variance β_t to the data.
- The backward (decoding) process trains the network to incrementally remove this noise, allowing the model to generate a natural sample x_0 out of pure noise (x_T).
- The process utilizes a logarithmic loss function relating the forward process q and learned reverse process p_θ :

$$J^{diff} = \log q(x_T | x_0) - \log p_\theta(x_0 | x_T).$$

4. Generative Adversarial Networks (GANs)

GANs introduce a paradigm for generating new data by pitting two distinct neural networks against one another in a Min-Max game. The goal of a GAN is to learn the true underlying data distribution to create highly realistic natural data samples.

The Two Adversaries

1. **The Generator (G):** Similar to a decoder, the Generator takes random noise ($\vec{\xi}$) as input and attempts to create synthetic data samples ($\vec{z}$). Its singular goal is to generate data realistic enough to fool the Discriminator.
2. **The Discriminator (D):** Acts as a binary classifier that takes an input image and predicts whether it is a "real" sample from the dataset or a "fake" sample created by the Generator.

The Min-Max Game and Loss Function

The Discriminator relies on Binary Cross-Entropy (BCE) loss. The overall objective is an adversarial competition: The Discriminator tries to maximize its accuracy in identifying real versus fake data, while the Generator tries to minimize the Discriminator's accuracy.

The combined loss function for the GAN is expressed as: $J^{GAN} = - \sum_{n=1}^N \log D(\vec{x}^{[n]}) - \sum_{n=1}^N \log(1 - D(G(\vec{\xi}^{[n]})))$

The ultimate goal of the Generator is mathematically defined as $G^* = \arg \max_G \min_D J^{GAN}$. The networks are updated via alternating training updating the weights of D first, and then the weights of G . Training is complete when the Generator creates data so realistic that the Discriminator is forced to guess blindly (outputting a probability of $y = 0.5$), after which the Discriminator is discarded.

Deep Convolutional GANs (DCGANs)

To generate complex images, standard fully-connected GANs are upgraded to DCGANs. Best practices for building DCGANs include:

- Using standard strides in the Discriminator and fractional strides in the Generator.
- Applying Batch Normalization after each layer (except the Generator's output and Discriminator's input).
- Using Leaky ReLU activation in the Discriminator, and standard ReLU in the Generator (with a $\tanh$ activation at the Generator's output).

5. Advanced GAN Architectures

Because base GANs take random noise as input, they have no understanding of classes or specific properties; they just generate random instances of the data distribution. Advanced architectures solve this.

Cycle GANs

Cycle GANs are used for image-to-image translation between two different visual domains without requiring paired training data (e.g., turning images of horses into zebras).

- They utilize two specific Generators ($G^{a \rightarrow b}$ and $G^{b \rightarrow a}$) and two specific Discriminators (D^a and D^b).
- **Cycle Consistency Loss:** Because the training data is unpaired, the network enforces "cycle consistency." This guarantees that if an image of a horse is translated into a zebra, and that resulting zebra is translated back, the final output closely matches the original horse. The cycle consistency loss equation is: $J^a = - \sum_{X^a \in I^a} |G^{b \rightarrow a}(G^{a \rightarrow b}(X^a)) - X^a|_1$.

Conditional GANs & Pix2Pix

Conditional GANs allow the user to dictate exactly what kind of image is generated by appending a class-specific condition or label ($\vec{c}$) to both the Generator's noise input and the Discriminator's input.

Pix2Pix is an application of Conditional GANs that focuses on strict image-to-image translation using *paired* data (e.g., turning a black-and-white photo into a color photo, or a medical MR image into a synthetic CT scan). The Pix2Pix loss function relies on a traditional GAN loss mechanism but adds an L1 distance penalty ($\lambda \sum ||X^{(b)} - G(X^{(a)}, \vec{\xi})||_1$) to guarantee that the generated output visually aligns with the specific target ground-truth image.

Lecture 9

1. The Motivation for Sequence Processing

Traditional deep learning architectures, such as standard fully-connected networks or Convolutional Neural Networks (CNNs), are designed to process inputs of a fixed size and dimension. However, real-world data often comes in the form of variable-length sequences. Examples include speech recognition (where people speak at different speeds), text processing and translation (where sentences vary in length), and video analysis like sign language recognition.

When processing text, characters or words are often represented using one-hot encoding, such as $a \Rightarrow (1, 0, 0, \dots, 0)$. A sequence is simply a collection of these encoded inputs over time, defined as $\vec{x}^1, \vec{x}^2, \dots, \vec{x}^S$.

Early Workarounds: Before Recurrent Networks, developers used workarounds to handle variable-length sequences:

- **Fixed-Size Padding:** Finding the longest sequence in a dataset and zero-padding all shorter samples to match that length before feeding them into a CNN.
- **Global Pooling:** Passing the variable-length sequence through convolutional layers and then applying global pooling to reduce the entire sequence down to a single, fixed-size mathematical representation.

The flaw here is that these methods fail to genuinely model the *sequential dependencies* of the data—where a current input strictly depends on the context of the inputs that preceded it.

2. Recurrent Neural Networks (RNNs)

Recurrent Neural Networks solve the sequence problem by introducing **feedback loops**. Instead of processing each input independently, an RNN maintains a hidden state that carries historical context forward.

- **The Elman Network (Simple RNN):** In this architecture, the output of a hidden neuron at step $s - 1$ is fed back as an input to the hidden neuron at step s . The hidden state is calculated using the current input and the previous hidden state: $\vec{a}^s = W^{(1)}\vec{x}^s + W^{(r)}\vec{h}^{s-1}$, followed by a non-linear activation $\vec{h}^s = g(\vec{a}^s)$.
- **The Jordan Network:** A variation where the *output* of the network ($\vec{y}^{s-1}$), rather than the hidden state, is fed back into the hidden layer for the next sequence item.

Unfolding in Time: To understand how an RNN learns, we conceptually "unfold" it. A recurrent loop over S time steps can be visualized as a deep, S -layer feed-forward network where the same weight matrices ($W^{(1)}$, $W^{(2)}$, and the recurrent weights $W^{(r)}$) are reused at every step.

3. Training & Backpropagation Through Time (BPTT)

RNNs are trained using a specialized gradient descent algorithm called Backpropagation Through Time (BPTT). However, BPTT exposes a critical flaw in simple RNNs: **Vanishing Gradients**.

Because the recurrent weight matrix $W^{(r)}$ is multiplied by itself at every step backward through time, the gradients tend to shrink exponentially. For a sequence of length S , the derivative involves $(W^{(r)})^{S-1}$. As a result, the network essentially "forgets" information from the earliest parts of the sequence because it learns very little from inputs like $\vec{x}^1$.

4. Gated Recurrent Networks

To fix the vanishing gradient problem and allow networks to learn long-term dependencies (such as grammar rules spanning an entire translated sentence), researchers introduced internal "gates".

Long Short-Term Memory (LSTM): LSTMs replace the simple recurrent matrix in an RNN with a complex block. They introduce a dedicated **memory cell** $\vec{c}$ and three distinct gates that learn how to manipulate that memory based on the input data:

1. **Forget Gate:** Decides what information from the previous memory $\vec{c}^{s-1}$ is no longer relevant and should be discarded.
2. **Input Gate:** Decides what new information from the current input $\vec{x}^s$ and previous hidden state $\vec{h}^{s-1}$ should be added to the memory.
3. **Output Gate:** Determines how the newly updated memory $\vec{c}^s$ influences the final hidden output $\vec{h}^s$. An LSTM block is highly parameterized, requiring the network to learn 8 different weight matrices to function.

Gated Recurrent Units (GRU): GRUs are a streamlined, simplified version of the LSTM. They completely remove the separate memory cell $\vec{c}$ and combine the gating mechanisms into just two gates: a **Reset Gate** and an **Update Gate**. Because of this simplification, GRUs are faster to train as they only require learning 6 weight matrices instead of 8.

5. Advanced Sequence Architectures

Convolutional LSTMs: When dealing with spatial-temporal sequences (like video frames or satellite imagery over time), standard fully-connected LSTM layers are computationally inefficient and ignore spatial relationships. Convolutional LSTMs solve this by replacing the standard matrix multiplications inside the gates with convolution operations.

Bidirectional RNNs (BiRNNs): Standard RNNs only have historical context. But sometimes, understanding a word in a sentence requires looking at the words that come *after* it. A BiRNN solves this by passing the sequence through two separate RNNs: one going forward, and one going backward. The outputs of both are combined to make predictions based on full surrounding context.

Sequence-to-Sequence (Seq2Seq) Modeling: In many tasks, the input length and output length differ drastically (e.g., summarizing a long document into a short paragraph).

- **Encoder:** An RNN reads the input sequence and compresses it into a final deep feature state $\vec{\phi}$.

- **Decoder (Autoregressive):** A second RNN takes $\vec{\varphi}$ as its initial state and generates the output sequence one step at a time. It feeds its own prediction at step s_y back in as the input for step $s_y + 1$ until the sequence is complete.

Attention Mechanisms: In highly complex sequence tasks, it's beneficial for the network to know which specific items in a sequence are most important. Attention gates compute an "attention score" α^s that dynamically weighs the importance of different sequence inputs, allowing the model to focus its resources on the most relevant contextual information.

Lecture 10

1. From RNNs to Transformers: Handling Sequences

Historically, sequential data was handled by processing elements step-by-step.

- **Recurrent Neural Networks (RNNs) & LSTMs:** Traditional models like Elman RNNs and Long Short-Term Memory (LSTM) networks process data forward through a sequence.
- **The Dependency Problem:** In these models, the current output depends on the current input and the hidden representation from the previous step. While LSTMs use complex gating mechanisms to model long-term dependencies, they still evaluate data sequentially and locally.
- **The Transformer Solution:** Transformers take all inputs and outputs at once, modeling global dependencies across the entire sequence. This means every input element can directly influence every output element, with the strength of that influence determined by "self-attention".

2. Core Mechanisms of the Transformer

The Transformer architecture shifts away from recurrence, relying entirely on attention mechanisms to draw global dependencies between inputs and outputs.

Self-Attention Calculation

Self-attention answers the question: *When evaluating a specific sequence element, how much attention should the model pay to all other elements?*

- **Query, Key, and Value:** The input is projected into three distinct vectors using learned weight matrices:
- **Query ($W^{(q)}$):** The attention target.
- **Key ($W^{(k)}$):** The attention source.
- **Value ($W^{(v)}$):** The attention content.
- **The Math:** Attention is calculated via a normalized dot product. The dot product between the Query and Key determines the alignment or "attention score". This score is scaled down by the square root of the dimension size (I) and passed through a SoftMax function to create a probability distribution:

$$a_{s',s} = \frac{e^{\hat{a}_{s',s}}}{\sum_{s''=1}^S e^{\hat{a}_{s',s''}}}$$

- **Final Output:** The final output is a weighted sum of the Value vectors, combined using the computed attention scores.

Network Architecture Additions

To make self-attention function effectively within a deep neural network, several other components are required:

- **Positional Encoding:** Because Transformers process everything at once, they have no inherent concept of sequence order. Positional encodings (often using sine and cosine functions) are added to the input embeddings so the model understands the relative position of tokens.
- **Multi-Headed Attention:** The network uses multiple sets of Query, Key, and Value weights in parallel. This allows the model to simultaneously attend to information from different representational spaces.
- **Class Token ([CLS]):** To gather global sequence information for tasks like classification, a separate, random input token called the [CLS] token is added. The final output corresponding to this token serves as an aggregate representation of the entire sequence.

3. Transformers Beyond Text: Vision and Sensor Data

While originally designed for Natural Language Processing, Transformers have been successfully adapted for images and sequential sensor data.

Vision Transformers (ViT) and SWIN

- **ViT:** The Vision Transformer scales images to a fixed size and chops them into patches (e.g., **16x16** pixels). These patches are linearly projected, combined with positional embeddings and a `[CLS]` token, and fed directly into a Transformer encoder.
- **SWIN Transformer:** To reduce the high computational complexity of standard global attention, the SWIN Transformer uses smaller patches (e.g., **4x4**) and applies local attention within shifted windows. Patches are merged progressively in deeper layers to create a hierarchical representation.

Sequential Sensor Data (Audio, Accelerometer, EEG)

Sensor data comes with high intra-class variability and high-frequency variations that are often uninterpretable by humans.

- **Audio/Sound:** Instead of processing raw waveforms, networks often use Mel Frequency Cepstral Coefficients (MFCCs). This involves using Short-Time Fourier Transform (STFT), taking the base-2 logarithm of the magnitude, and applying a human-inspired log-based Mel-scale. These MFCCs can then be treated sequentially (via RNNs/Transformers) or as a 2D image (via CNNs/ViTs).
- **Multi-Channel Recordings (e.g., EEG):** When dealing with multiple channels (like brain waves or car sensors), standard self-attention becomes computationally heavy (scaling quadratically across tokens and channels).
- **Separable Self-Attention:** A solution is to separate the attention into an intra-channel block (attention within a single channel over time) and an inter-channel block (attention across different channels at a specific time).

4. Foundation Models

Foundation models address the challenge that labeling data is expensive and prone to error, whereas gathering unlabeled data (like internet text or unannotated images) is easy. The core idea is to train a massive domain-specific model on unlabeled data, then fine-tune it for specific downstream tasks.

Masked Auto-Encoders (MAE)

MAE is a self-supervised learning technique.

- **Image MAE:** The model randomly masks a high ratio of input patches (e.g., **75%**). The encoder only processes the visible, non-masked inputs. A decoder then uses learnable `[MASK]` tokens to attempt to reconstruct the original missing pixels. The loss function (J^{L_2}) is computed exclusively on the masked pixels.
- **EEG MAE:** This concept adapts to sensor data by masking input patches with a single learnable mask token and padding varying numbers of input channels.

Contrastive Language-Image Pretraining (CLIP)

CLIP bridges the gap between text and visual data by training on vast amounts of image-caption pairs.

- **Architecture:** It uses a pre-trained image encoder (like ResNet or ViT) to extract image features and a text encoder (like BERT or GPT) to extract text features. Both feature vectors are projected into a shared embedding space.
- **Contrastive Learning:** The model computes the cosine similarity between the image and text embeddings. Using a SoftMax function across the batch, it optimizes the model so that matching image-text pairs yield high similarity scores, while non-matching pairs yield low scores.
- **Zero-Shot Classification:** CLIP can be used for classification without fine-tuning. By generating text prompts (e.g., "A photo of a dog"), generating a text embedding, and comparing it to an image embedding, the model simply selects the class with the highest similarity score.

Lecture 11

1. Multitask Learning & Multi-Objective Networks

Multitask Learning (MTL) is designed to solve multiple related tasks simultaneously by leveraging the commonalities between them. Instead of training separate models, an MTL network uses a shared architectural backbone that branches out into different outputs.

- **The Loss Function:** The network is optimized using a combined loss function: $J^{total} = \lambda_1 J^1 + \lambda_2 J^2 + \dots$. A major challenge in MTL is finding appropriate weights (λ_i) to ensure the losses are balanced and one task does not dominate the others.
- **Dataset Bias and Imbalance:** The lecture uses the CelebA dataset—which contains 40 binary facial attributes—as a case study. While attributes like "Attractive" are relatively balanced, others like "Bald" or "Chubby" are highly imbalanced.
- **The Problem with Naïve Training:** If you train this network using standard binary cross-entropy, the model tends to learn the dataset bias. For minority classes, the network simply predicts the majority class to minimize loss, effectively failing to learn the actual feature.
- **Weighted Loss & Grad-CAM:** Because standard oversampling or undersampling is impossible for 40 simultaneous outputs, the solution is applying a weighted loss function (e.g., J^{wL_2}). Using Class Activation Mapping (Grad-CAM), we can visualize that imbalanced classifiers often look at the background to guess majority classes, whereas balanced classifiers focus on reasonable facial regions.

2. Advanced Data Augmentation

Data augmentation creates artificial samples via transformations (like shifting, scaling, rotating, and cropping) to prevent overfitting.

- **Test-Time Augmentation (TTA):** Unlike standard augmentation that draws random transformations every epoch during training, TTA applies a *fixed* set of augmentations to an image during the testing phase. The network predicts outcomes for all variants, and the final result is aggregated (e.g., via weighted sums or evolutionary algorithms).
- **Alignment-Free Facial Attribute Classification (AFFECT):** Traditionally, facial classification required error-prone landmark detection to align faces. By applying heavy spatial perturbations during training, the network becomes inherently stable against misalignments, rendering facial landmark detection obsolete while maintaining high accuracy.

3. Object Detection and Segmentation

Object detection involves two distinct tasks: **localization** (predicting bounding boxes) and **classification** (identifying the object inside the box).

Object Detection Models

- **Two-Stage Detectors (Fast/Faster R-CNN):** These models first generate region proposals. Fast R-CNN relies on external selective search algorithms, while Faster R-CNN integrates a fully-convolutional Region Proposal Network (RPN) directly into the architecture.
- **One-Stage Detectors (YOLO & SSD):** Models like YOLO (You Only Look Once) divide the input into a grid, where each cell simultaneously predicts object existence, bounding boxes, and classes. SSD (Single Shot MultiBox Detector) extends this concept by extracting information from several convolutional layers at different resolutions to better detect objects of varying sizes.

Image Segmentation

Segmentation requires pixel-level labeling across an image.

- **Mask R-CNN:** Extends Faster R-CNN by adding a parallel mask branch that predicts an $O \times 14 \times 14$ mask for each region proposal.
- **U-Net:** A Fully-Convolutional Network (FCN) featuring a symmetrical encoder-decoder structure. It uses "skip connections" (copy and crop) to preserve high-resolution spatial details lost during pooling, optimizing a per-pixel weighted cross-entropy loss.

4. Alternative Architectural Approaches

Radial Basis Function (RBF) Networks

Instead of traditional dot-product activations, RBF networks rely on spatial proximity.

- **Distance-based activation:** $a_k = |\vec{w}_k - \vec{x}|$.
- **Centered activation:** This distance is passed through a Gaussian function: $h_k = e^{-\frac{a_k^2}{2\sigma_k^2}}$. The network automatically learns spatial centers, but it is rarely used as a standalone architecture today.

Visual Attention Models

These models selectively emphasize informative features within Convolutional Neural Networks.

- **Squeeze-and-Excitation (SE) Block:** Uses global pooling to "squeeze" spatial channels, then applies fully-connected layers and a Sigmoid activation to dynamically weight the importance of each channel.

- **CBAM (Convolutional Block Attention Module):** Computes both Channel Attention (what features matter) and Spatial Attention (where they matter using a 7×7 convolution) sequentially.

Siamese Networks

Siamese networks map two distinct inputs ($\vec{x}_1, \vec{x}_2$) into an embedding space to learn discriminative features based on similarity. * **Contrastive Learning:** Utilizes a margin ρ in the loss function (J^{CL}) to force positive pairs to be highly similar while ensuring negative pairs are separated by at least the margin.

- **Triplet Loss:** Evaluates an Anchor, a Positive, and a Negative sample concurrently. The network is penalized unless the distance between the anchor and the negative is greater than the distance between the anchor and the positive plus a margin ρ .
- **Hard Negative Mining:** To keep the network learning efficiently, the algorithm actively searches for the most difficult negative samples to evaluate against the anchor.

Graph Convolutional Networks (GCNs)

GCNs are designed for non-Euclidean data structures like networks, where entities are represented as nodes and relationships as edges (adjacency matrix).

- **Goal:** Semi-supervised node classification—predicting missing labels under the assumption that connected nodes share similar characteristics.
- **Spectral Graph Convolution Layer:** Information is propagated between nodes using the layer equation: $H^{[n]} = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} X^{[n]} W$. Here, $\tilde{A} = A + I$ injects "self-excitation" so nodes retain their own features, and $\tilde{D}$ is the degree matrix.

Lecture 12

1. Fundamentals and Loss Functions

- **Deep Learning Elements:** Training deep networks requires a large dataset of inputs (X) and corresponding labels (T). It also requires a parametrized model ($f_{\Theta}(X)$) to generate predictions (Y). Additionally, a loss function ($\mathcal{J}(Y, T)$) is needed to measure the prediction error, along with a learning algorithm to adapt parameters.
- **Regression:** This is used for continuous targets. It frequently employs the Mean Squared Loss function, mathematically defined as $J^{L_2} = \sum_{n=1}^N (\vec{y}^{[n]} - \vec{t}^{[n]})^2$.
- **Binary Classification:** This handles two-class problems where the outcome is either 0 or 1. It utilizes the Binary Cross-Entropy loss function (J^{BCE}).
- **Categorical Classification:** This is applied to multi-class problems. Outcomes are represented using one-hot vectors. The model is trained using the Categorical Cross-Entropy loss function (J^{CCE}).
- **Open-Set Classification:** This expands upon categorical classification by introducing the capacity to reject inputs belonging to unknown classes. Unknown samples are represented with a uniform target distribution. Models are evaluated using an Open-Set Classification Rate (OSCR) curve.

2. Layer Types

- **Fully-Connected Layers:** These compute outputs via a simple matrix multiplication formula, expressed as $\vec{a} = W \vec{x}$.
- **Radial Basis Function (RBF) Layers:** These compute outputs based on the distance between input vectors and the weights. The computation follows the formula $a_k = \exp(-\|\vec{w}_k - \vec{x}\|^2)$.
- **Convolutional Layers:** These operate across 1D, 2D, or 3D data forms to extract contextual features. Strided convolutions are used to reduce data dimensions, while zero-padding helps retain original dimensions. Transposed convolutions are utilized to upsample or increase dimensions.
- **Dilated Convolutions:** These introduce gaps between offset points in the convolutional filter. This increases the receptive field without increasing the total number of learnable parameters.
- **Deformable Convolutions:** These layers learn non-integral spatial offset locations ($(\vec{\kappa}_{u,v})$). This allows the kernel shape to dynamically adapt to the features present in the input.
- **Pooling Layers:** These perform spatial downsampling. Standard methods include average pooling and maximum pooling.
- **Sequence Processing Layers:** Recurrent Neural Networks (RNNs) map sequences using feedback loops. Advanced blocks like Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) contain specialized gates to control information flow over time.

- **Self-Attention Layers:** These layers learn the varying importance of different elements within a sequence. This mechanism is the foundational building block of Transformer networks.

3. Activation Functions

- **Sigmoid and Tanh:** The logistic sigmoid function maps inputs to a range between 0 and 1. The hyperbolic tangent function maps inputs to a range between -1 and 1.
- **Gaussian:** This provides a centered activation function commonly used in RBF networks.
- **ReLU Variants:** The standard Rectified Linear Unit (*ReLU*) outputs zero for negative values. Variations include Leaky ReLU, which applies a fixed minor slope for negative values, and Parametric ReLU, which learns this slope during training.
- **SoftMax:** This is typically applied at the network's output layer. It converts raw logits ($\vec{z}$) into normalized categorical probabilities ($\vec{y}$).
- **MaxOut:** This activation learns multiple parts for a layer and outputs the maximum value among them.
- **SPLASH:** This is a learnable activation function. It features adaptive capabilities utilizing symmetric hinges.

4. Network Structures

- **Shallow Networks:** The simplest form is the Perceptron, which uses a non-differentiable Heaviside step activation. Two-layer networks are fundamental because the Universal Approximation Theorem states they can approximate any continuous function given enough neurons.
- **Deeper Networks:** Early deep learning successes included LeNet and AlexNet. Subsequent architectures like VGG relied on stacked 3×3 convolutions. Residual Networks (ResNets) introduced stacked residual bottleneck blocks to allow training of significantly deeper networks without vanishing gradients.
- **Encoder-Decoder Structures:** Auto-Encoders consist of an encoder that compresses data into a low-dimensional embedding ($\vec{\phi}$) and a decoder that attempts to reconstruct the original input.
- **Transformer Networks:** These rely purely on multi-head attention blocks without recurrence. Pretrained generative examples include models like ChatGPT.
- **Vision Transformers (ViT):** These apply the transformer architecture directly to image recognition tasks. They accomplish this by flattening standard image patches into linear sequences.
- **Inception Modules:** These architectures perform parallel operations of varying sizes, such as 1×1 , 3×3 , and 5×5 convolutions. They often rely on factorized convolutions to reduce parameter count.
- **DenseNet:** In this structure, the output feature maps of a layer serve as direct inputs to all subsequent layers within a dense block.
- **Squeeze and Excitation Networks:** These networks utilize global pooling and two fully-connected layers to automatically compute the importance of individual feature channels. They then scale the channels by these importance weights.

5. Learning Algorithms

- **Gradient Descent:** The primary goal is to minimize the loss. Parameters are updated by taking steps in the direction of the negative gradient, calculated as $\vec{w} = \vec{w} - \eta \nabla_{\vec{w}} J$.
- **Stochastic Gradient Descent (SGD) and Adam:** SGD partitions the training dataset into smaller batches to perform weight updates. The Adam optimizer introduces an adaptive learning rate by maintaining running averages of gradients and their squares.
- **Error Backpropagation:** Gradients are computed through layered functions using the chain rule. This relies heavily on calculating Jacobian matrices.
- **Adversarial Samples:** Using methods like Fast Gradient Sign (FGS), network predictions can be manipulated. This is done by calculating the derivative of the loss with respect to the input image and introducing targeted noise.
- **Class Activation Mapping (CAM):** This calculates the partial derivative of the output logit with respect to the final convolutional activation map. This allows the visualization of the regions the network deemed most important for its decision.
- **Weakly-Labeled & Unsupervised Learning:** Contrastive learning models, such as CLIP, train by comparing positive and negative pairings of images and text. Generative approaches include Variational Auto-Encoders (VAEs), Diffusion Models, and Generative Adversarial Networks (GANs). Masked Auto-Encoders (MAE) are used in foundational models to learn representations by predicting missing patches.

6. Regularization Techniques

- **Weight Decay:** Limits the overall magnitude of the network's weights to prevent overfitting. This is achieved by adding an L_1 or L_2 penalty term to the primary loss function.

- **DropOut:** This technique randomly zeros out neuronal activations during training with a specified probability (κ).
- **Batch Normalization:** Standardizes a layer's outputs by maintaining a running mean and variance to normalize data to a normal distribution ($N_{0,1}$).



Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 1



ansar-butt

Languages

● C++ 100.0%

Suggested workflows

Based on your tech stack



CMake based, multi-platform projects

Build and test a CMake based project on multiple platforms.

Configure



C/C++ with Make

Build and test a C/C++ project using Make.

Configure



MSBuild based projects

Build a MSBuild based project.

Configure

[More workflows](#)

[Dismiss suggestions](#)